Asynchronous and Parallel Programming

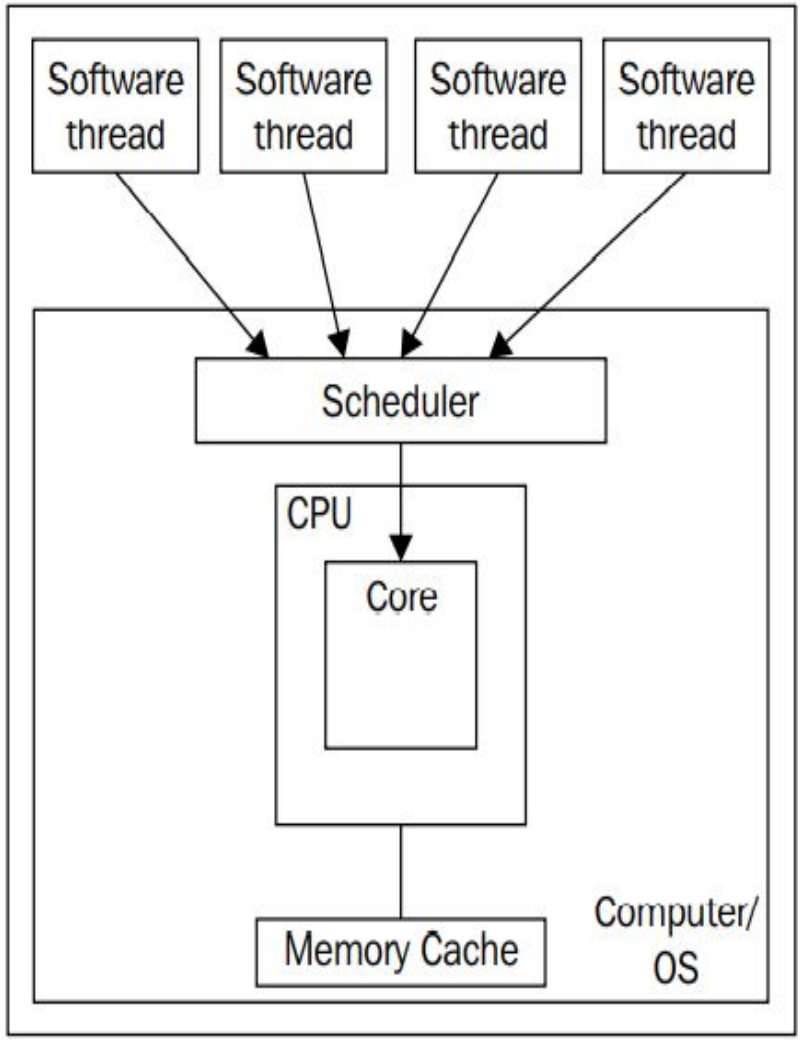
Objectives

- Overview Mono-Processor Systems and Multiprocessor Systems
- Overview Multiple Core Processors and Hyper-Threading
- Overview Flynn's Taxonomy
- Describe about Serial Computing
- Describe about Parallel Computing
- Overview The Parallel Programming Architecture
- Overview Task Parallel Library (TPL) and Parallel LINQ (PLINQ)
- Overview Asynchronous Programming in .NET
- Demo about Task Parallel Library (TPL) and Parallel LINQ (PLINQ)
- Demo about Asynchronous Programming by async and await keywords

Introduction to Parallel Computing

Understanding Mono-Processor Systems

- The mono-processor systems use old-fashioned, classic computer architecture (developed by the outstanding mathematician, John von Neumann, in 1952)
- The microprocessor receives an input stream, executes the necessary processes, and sends the results in an output stream that is distributed to the indicated destinations
- The beside diagram represents a mono-processor system (one processor with just one core) with one user and one task running
- This working scheme is known as input-processing-output (IPO) or single instruction, single data (SISD)
- The von Neumann's bottleneck problem (delay)

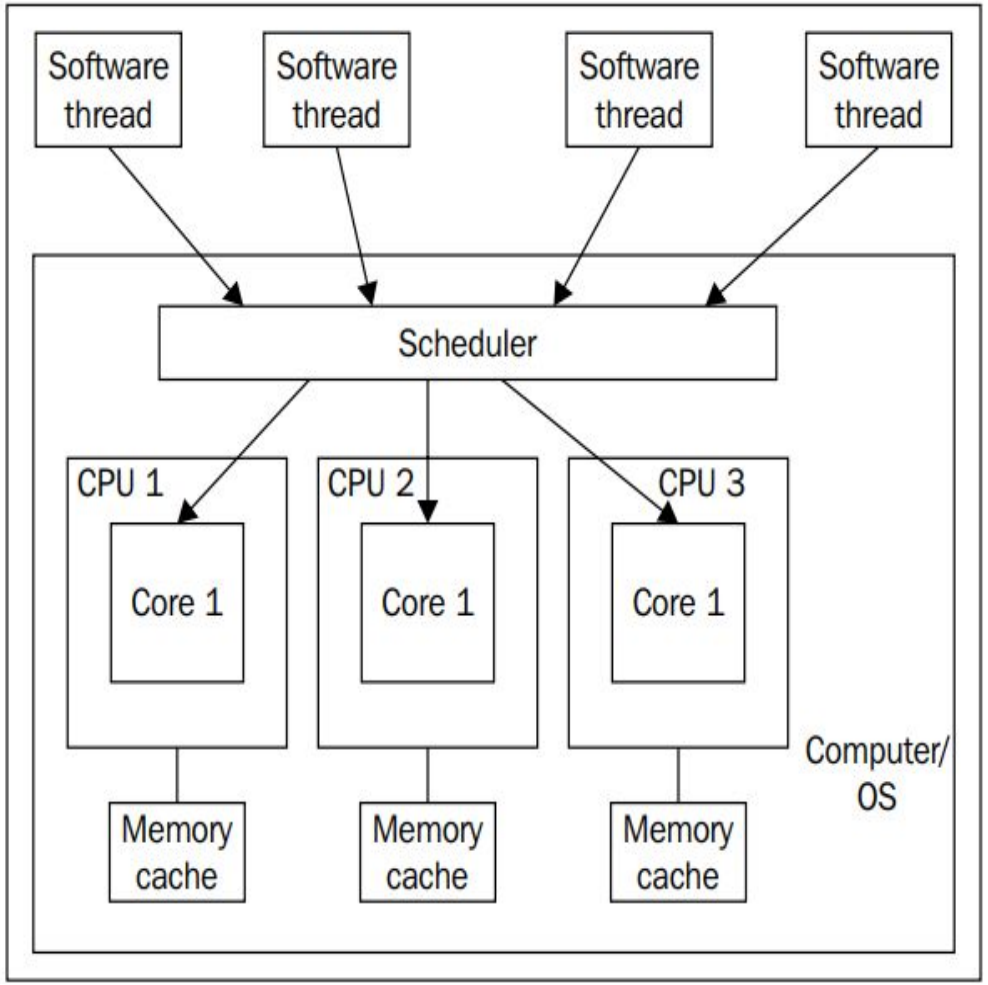


Understanding Multiprocessor Systems

- Systems with multiple processors are a solution to von Neumann's bottleneck
- There are two basic procedures to distribute tasks in systems with multiple processors:
  - **Symmetrical multiprocessing (SMP)**: Any available processor or core can execute tasks. The most used and efficient one is n-way symmetrical multiprocessing, where n is the number of installed processors. With this procedure, each processor can execute a task isolated from the rest and also when a particular software is not optimized for multiprocessing systems
  - **Asymmetrical multiprocessing (AMP or ASMP)**: Usually, one processor acts as the main processor. It works as a manager and is in charge of distributing the tasks to the other available processors, using different kinds of algorithms for this purpose

Understanding Multiprocessor Systems

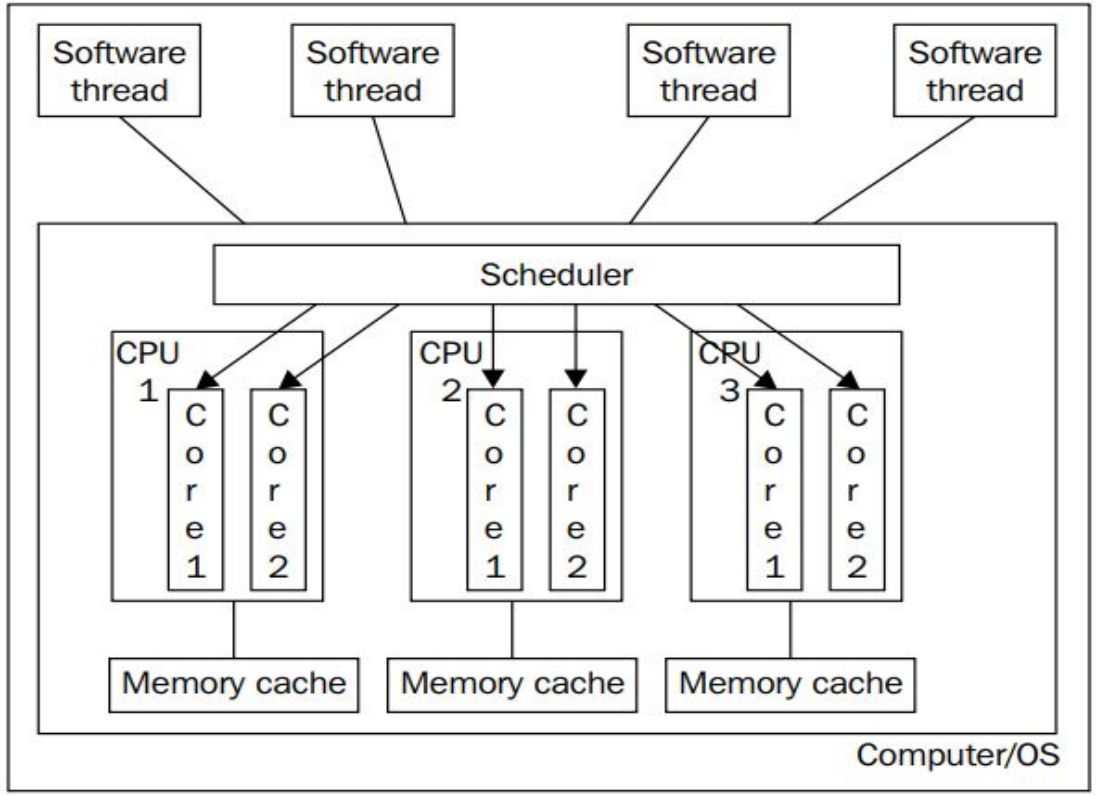
- The n-way symmetric multiprocessing procedure achieves the best performance and the best resources usage, where n can be two or more processors
- A symmetric multiprocessing system with many users connected or numerous tasks running provides a good solution to von Neumann's bottleneck. The multiple input streams are distributed to the different available processors for their execution, and they generate multiple concurrent output streams, as shown in the beside diagram



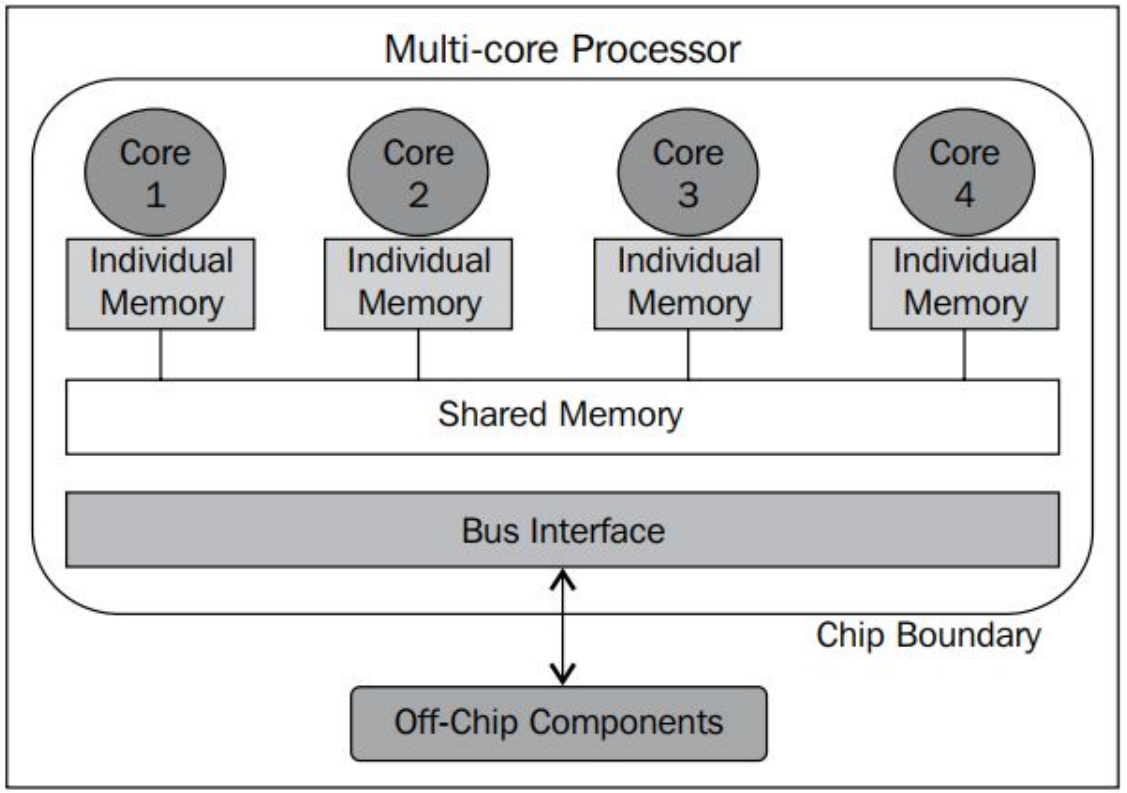
Multiple Core Processors

- A multiple core CPU has more than one physical processing unit. In essence, it acts like more than one CPU. The only difference is that all cores of a single CPU share the same memory cache instead of having their own memory cache
- The total number of cores across all of the CPUs of a system is the number of physical processing units that can be scheduled and run in parallel, that is, the number of different software threads that can truly execute in parallel
- There is a slight performance bottleneck with having multiple cores in a CPU versus having multiple CPUs with single cores due to the sharing of the memory bus (for most applications, this is negligible)
- For the parallel developer trying to estimate performance gains by using a parallel design approach, the number of physical cores is the key factor to use for estimations

Multiple Core Processors



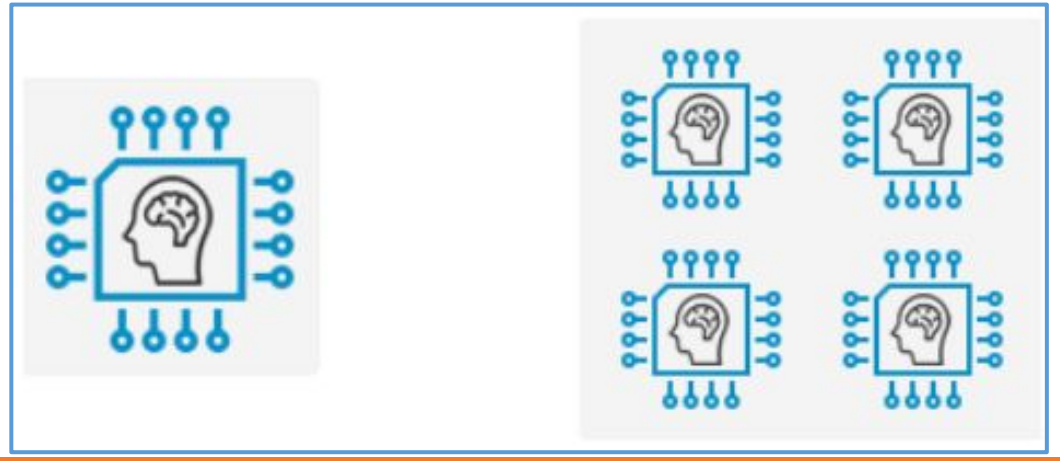
The diagram shows three physical CPUs each having two logical cores



The diagram shows a CPU with four logical core each having its own memory and then shared memory between thems

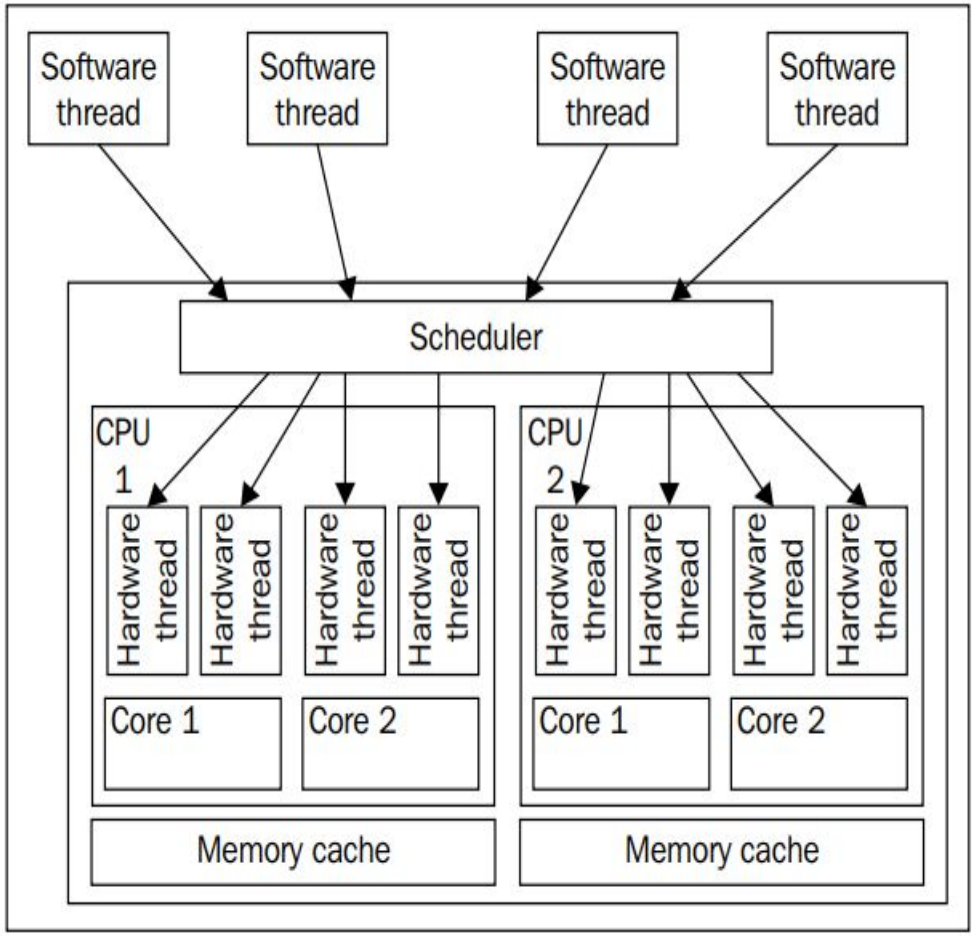
Hyper-Threading (HT)

- Hyper-threading (HT) technology is a proprietary technology that was developed by Intel that improves the parallelization of computations that are performed on x86 processors
- It was first introduced in Xeon server processors in 2002. HT enabled single processor chips run with two virtual (logical) cores and are capable of executing two tasks at a time
- The following diagram shows the difference between single and multi-core chips:



Hyper-Threading

- Each of these logical cores is called a hardware thread and can be scheduled separately by the operating system (OS) scheduler
- Even though each hardware thread (logical core) appears as a separate core for the OS to schedule, only one logical core per physical core can execute a software instruction at a time

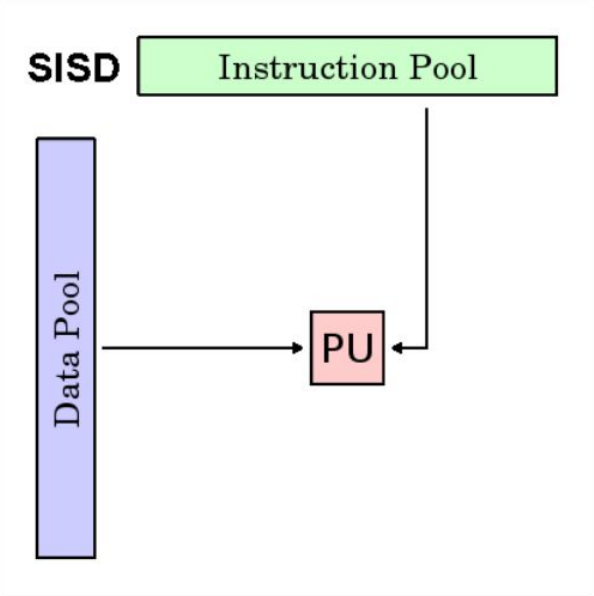


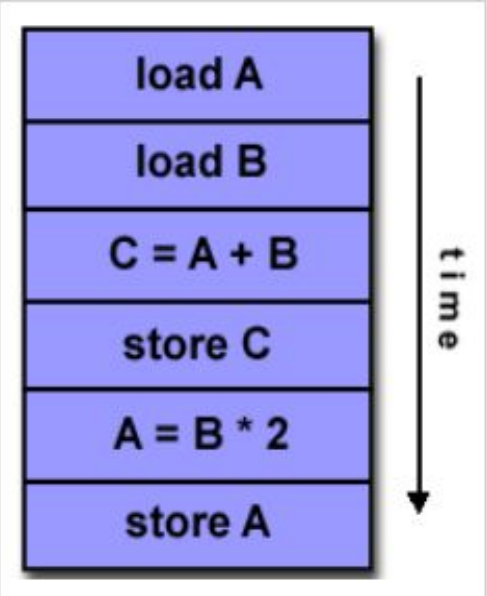
Hyper-Threading

- The following are a few examples of processor configurations and the number of tasks that they can perform:
  - **A single processor with a single-core chip**: One task at a time
  - **A single processor with an HT-enabled single-core chip**: Two tasks at a time
  - **A single processor with a dual-core chip**: Two tasks at a time
  - **A single processor with an HT-enabled dual-core chip**: Four tasks at a time
  - **A single processor with a quad-core chip**: Four tasks at a time
  - **A single processor with an HT-enabled quad-core chip**: Eight tasks at a time

Understanding Flynn's Taxonomy

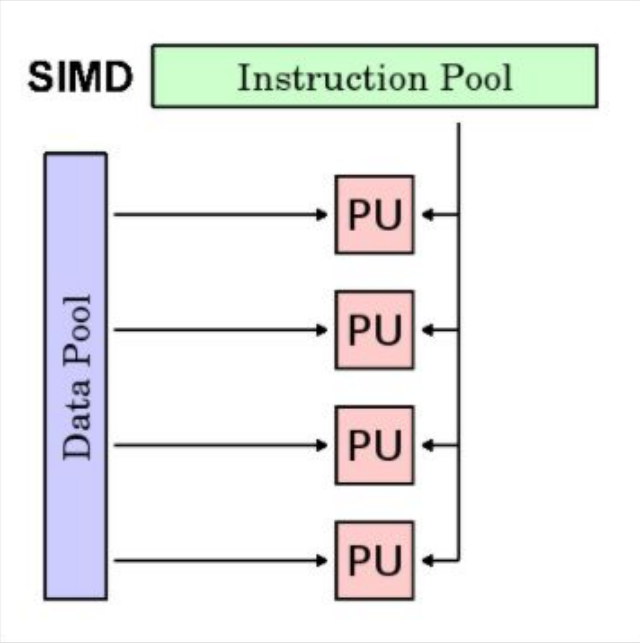
- Flynn classified computer architectures into four categories based on the number of concurrent instruction (or control) streams and data streams:
  - **Single Instruction, Single Data (SISD)**: In this model, there is a single control unit and a single instruction stream. These systems can only execute one instruction at a time without any parallel processing. All single-core processor machines are based on the SISD architecture

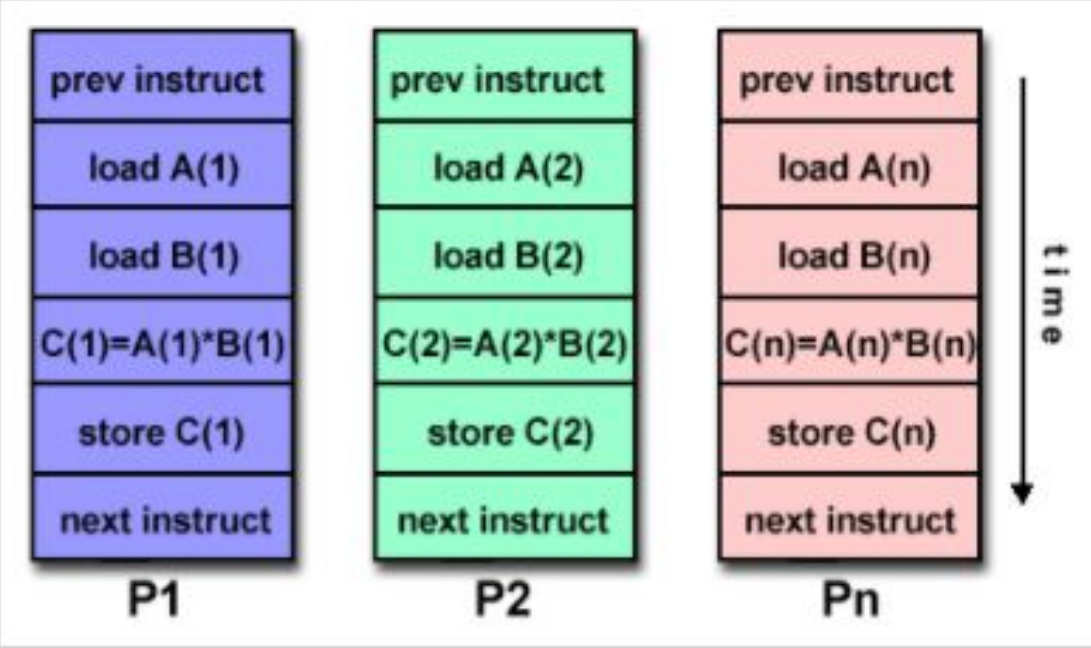




Understanding Flynn's Taxonomy

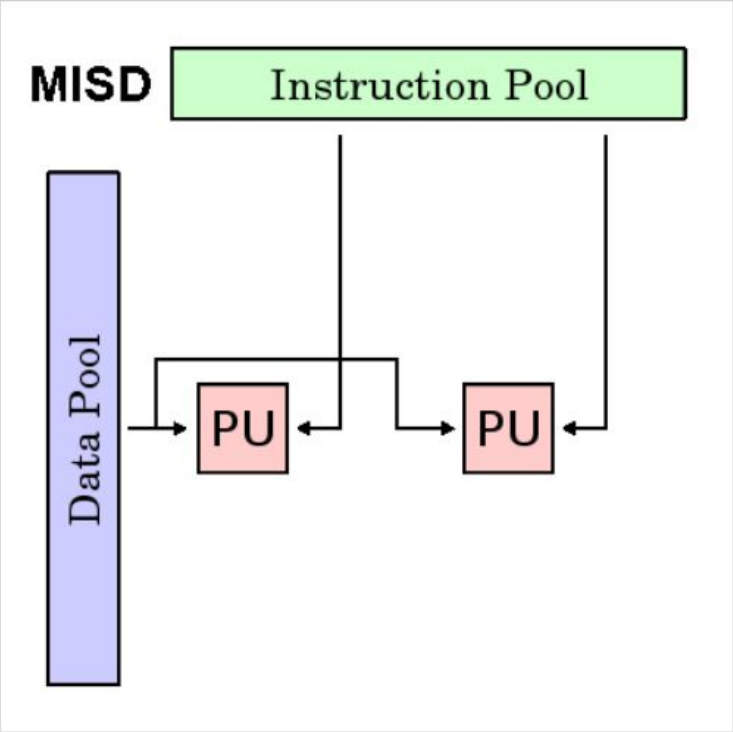
- **Single Instruction, Multiple Data (SIMD)**: In this model, we have a single instruction stream and multiple data streams. The same instruction stream is applied to multiple data streams in parallel. This is handy in speculativeapproach scenarios where we have multiple algorithms for data and we don't know which one will be faster. It provides the same input to all the algorithms and runs them in parallel on multiple processors

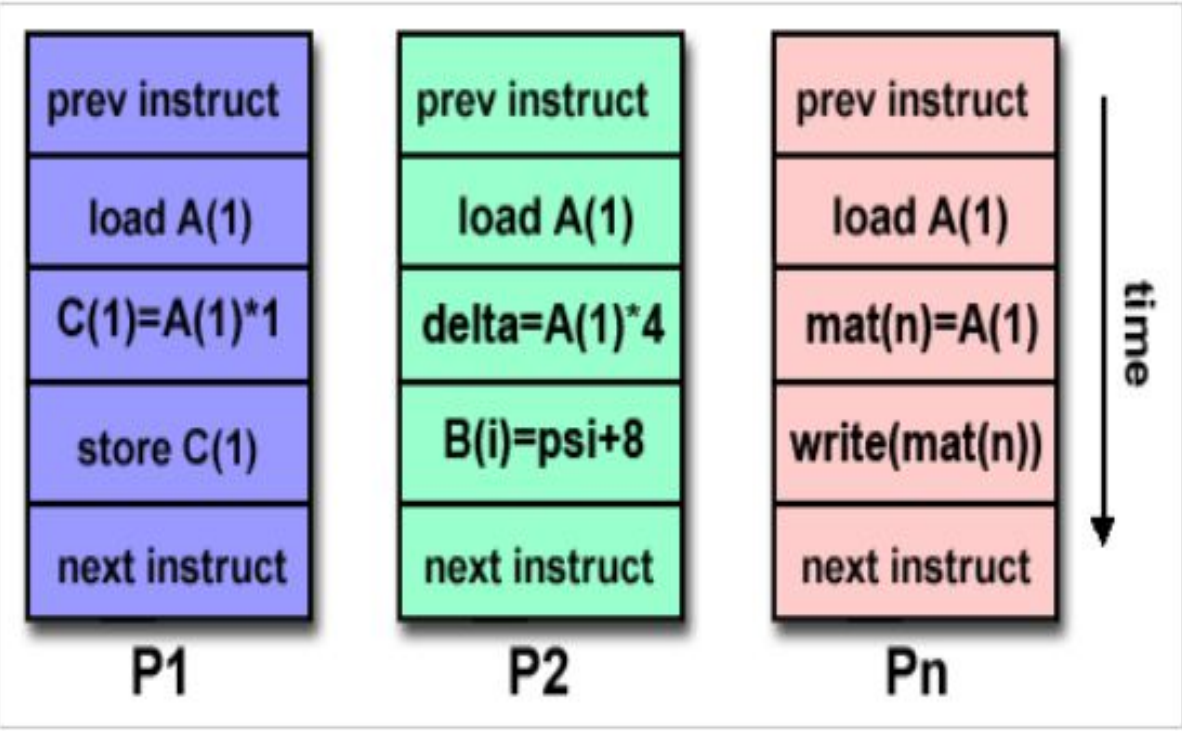




Understanding Flynn's Taxonomy

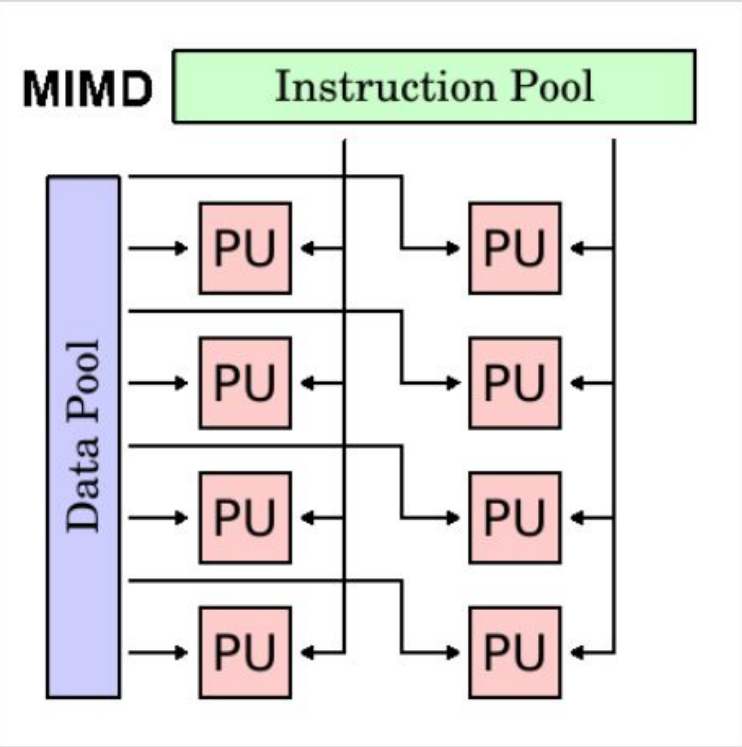
- **Multiple Instructions, Single Data (MISD)**: In this model, multiple instructions operate on one data stream. Therefore, multiple operations can be applied in parallel on the same data source. This is generally used for fault tolerance and in space shuttle flight control computers

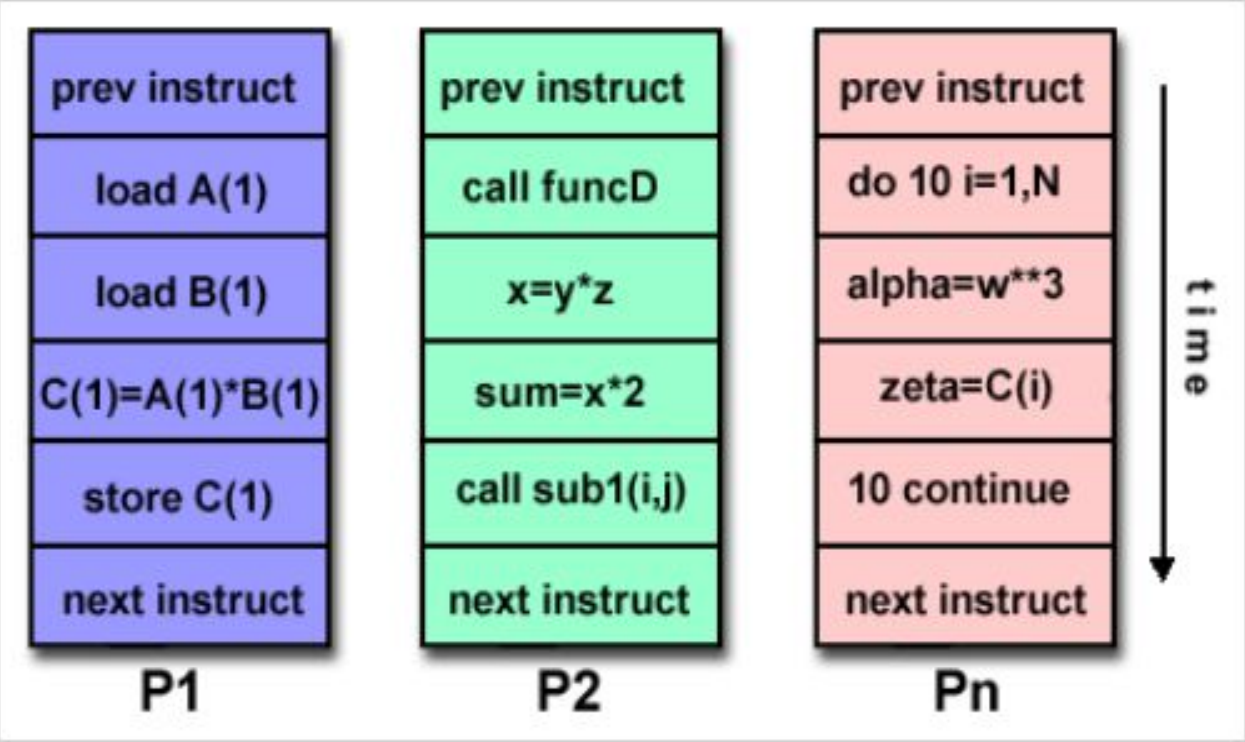




Understanding Flynn's Taxonomy

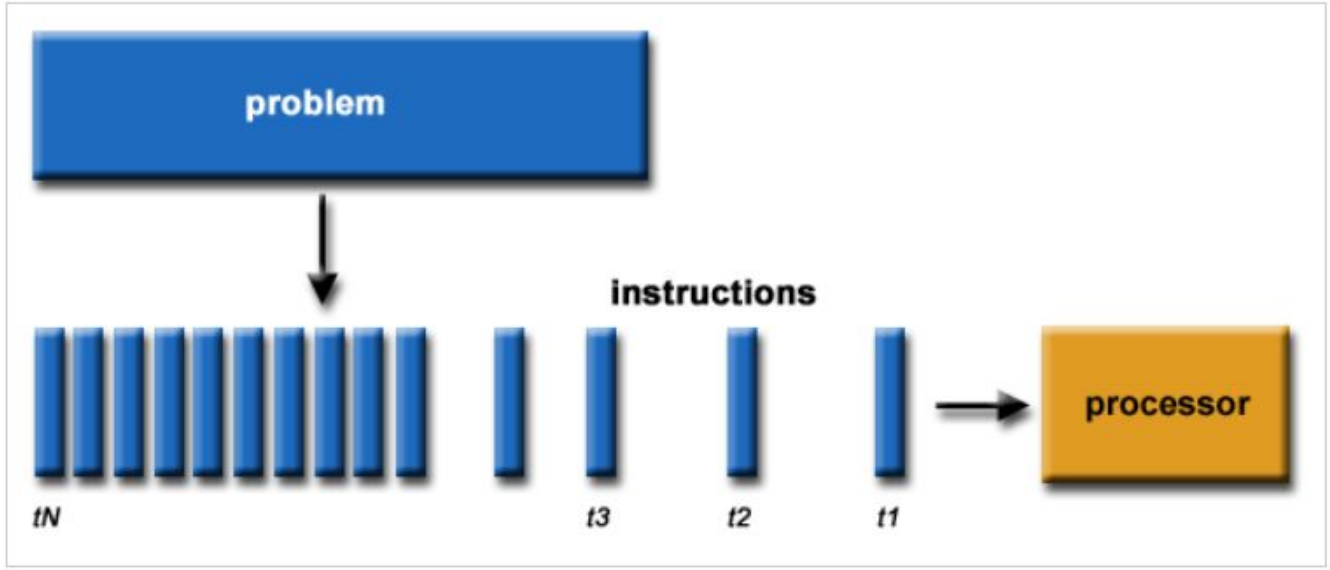
- **Multiple Instructions, Multiple Data (MIMD)**: In this model, as the name suggests, we have multiple instruction streams and multiple data streams. Due to this, we can achieve true parallelism, where each processor can run different instructions on different data streams. Nowadays, this architecture is used by most computer systems





Understanding Serial Computing

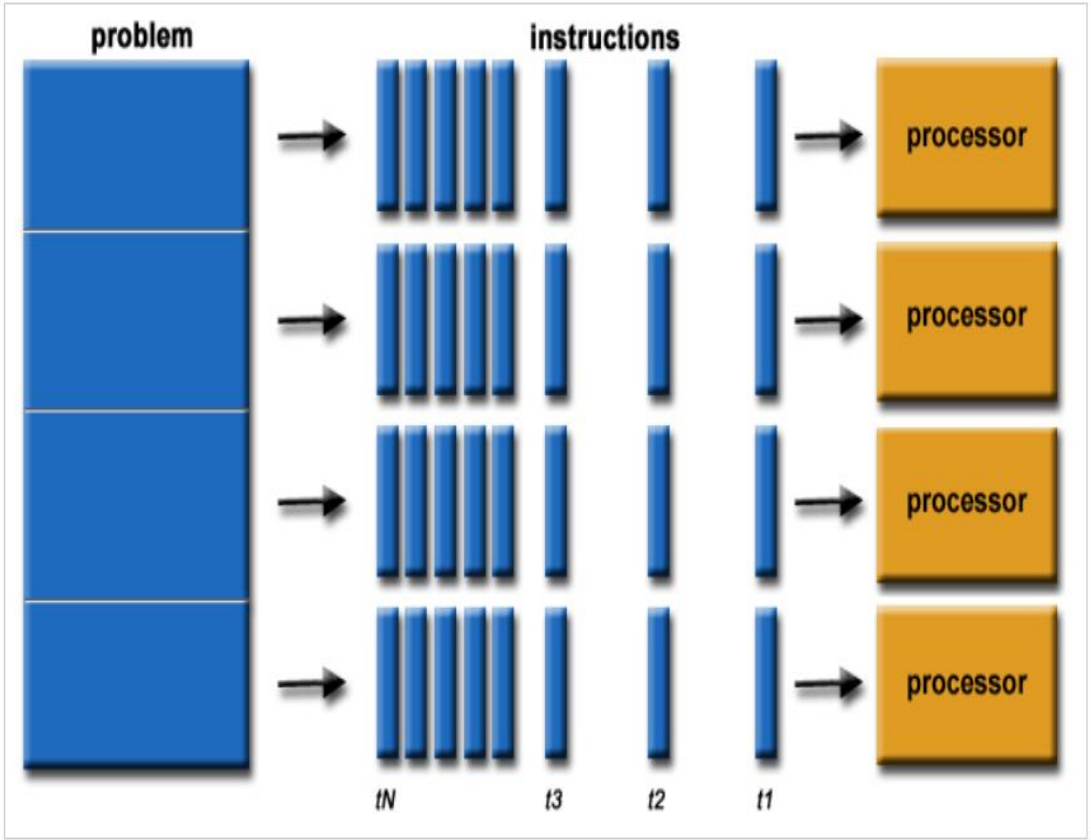
- Traditionally, software has been written for **serial** computation:
  - A problem is broken into a discrete series of instructions
  - Instructions are executed sequentially one after another
  - Executed on a single processor
  - Only one instruction may execute at any moment in time



Understanding Parallel Computing

In the simplest sense, **parallel computing** is the simultaneous use of multiple compute resources to solve a computational problem:

- A problem is broken into discrete parts that can be solved concurrently
- Each part is further broken down to a series of instructions
- Instructions from each part execute simultaneously on different processors
- An overall control/coordination mechanism is employed



Understanding Parallel Computing

- Advantages of **Parallel** Computing over **Serial** Computing are as follows:
  - It saves time and money as many resources working together will reduce the time and cut potential costs
  - It can be impractical to solve larger problems on Serial Computing
  - It can take advantage of non-local resources when the local resources are finite
  - Serial Computing ‘wastes’ the potential computing power, thus Parallel Computing makes better work of the hardware
- Type of Parallel
  - Bit-level parallelism
  - Instruction-level parallelism
  - Data-level parallelism (DLP)
  - Task Parallelism

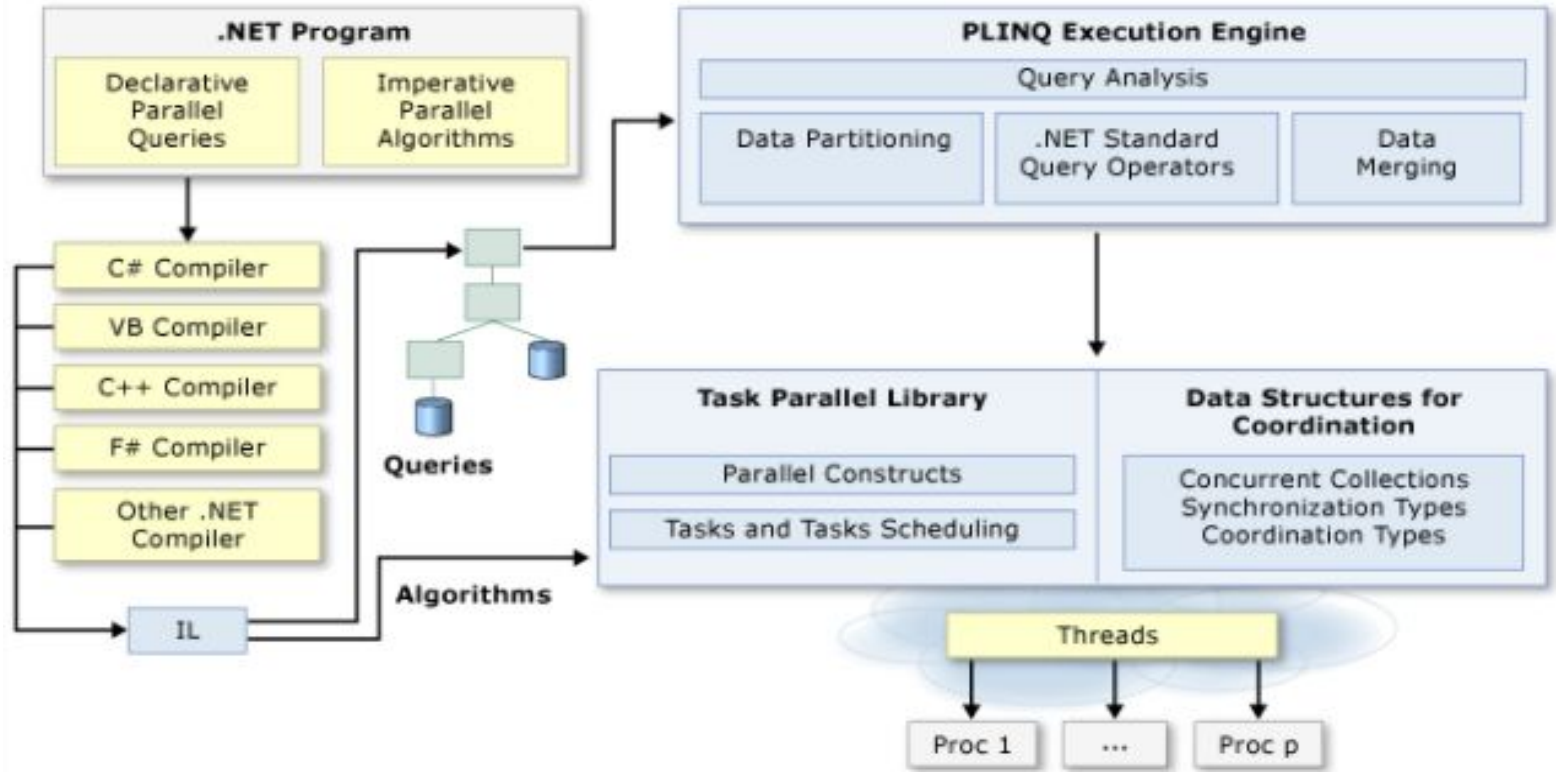
Understanding Parallel Computing

- Applications of Parallel Computing:
  - Databases and Data mining
  - Real-time simulation of systems
  - Science and Engineering.
  - Advanced graphics, augmented reality, and virtual reality
- Limitations of Parallel Computing:
  - It addresses such as communication and synchronization between multiple sub-tasks and processes which is difficult to achieve
  - The algorithms must be managed in such a way that they can be handled in a parallel mechanism
  - The algorithms or programs must have low coupling and high cohesion. But it's difficult to create such programs
  - More technically skilled and expert programmers can code a parallelism-based program well. Advanced graphics, augmented reality, and virtual reality

Parallel Programming in .NET

The Parallel Programming Architecture

- Visual Studio and .NET enhance support for parallel programming by providing a runtime, class library types, and diagnostic tools (introduced in .NET Framework 4) simplify parallel development
- We can write efficient, fine grained, and scalable parallel code in a natural idiom without having to work directly with threads or the thread pool



Understanding Task Parallel Library (TPL)

- The Task Parallel Library (TPL) is a set of public types and APIs in the System.Threading and System.Threading.Tasks namespaces
- The purpose of the TPL is to make developers more productive by simplifying the process of adding parallelism and concurrency to applications
- The TPL scales the degree of concurrency dynamically to most efficiently use all the processors that are available. In addition, the TPL handles the partitioning of the work, the scheduling of threads on the ThreadPool, cancellation support, state management, and other low-level details
- By using TPL, we can maximize the performance of our code while focusing on the work that our program is designed to accomplish

System.Threading.Tasks Namespace

- Provides types that simplify the work of writing concurrent and asynchronous code
- The main types are **Task** which represents an asynchronous operation that can be waited on and cancelled, and **Task<TResult>**, which is a task that can return a value
- The **TaskFactory** class provides static methods for creating and starting tasks, and the **TaskScheduler** class provides the default thread scheduling infrastructure
- Tasks provide features such as await, cancellation, and continuation, and these run after a task has finished
- The following table describes some of the key classes:

System.Threading.Tasks Namespace

Class Name	Description
Parallel	Provides support for parallel loops and regions
Task	Represents an asynchronous operation
Task<TResult>	Represents an asynchronous operation that can return a value
TaskFactory	Provides support for creating and scheduling Task objects
TaskFactory<TResult>	Provides support for creating and scheduling Task<TResult> objects
TaskScheduler	Represents an object that handles the low-level work of queuing tasks onto threads
TaskAsyncEnumerableExtensions	Provides a set of static methods for configuring task-related behaviors on asynchronous enumerables and disposables
TaskCanceledException	Represents an exception used to communicate task cancellation
TaskCompletionSource	Represents the producer side of a Task unbound to a delegate, providing access to the consumer side through the Task property

The System.Threading.Tasks.Task class

- A Task class is a way of executing work asynchronously as a ThreadPool thread and is based on the Task-Based Asynchronous Pattern (TAP)
- The non-generic Task class doesn't return results, so whenever we need to return values from a task, we need to use the generic version, Task<T>
- We can create a task using the Task class in various ways:
  - Using lambda expressions syntax

```
Task task = new Task(() => PrintNumber10Times());
task.Start();
```

  - Using the Action delegate

```
Task task = new Task(new Action(PrintNumber10Times));
task.Start();
```

  - Using delegate

```
Task task = new Task(delegate{ PrintNumber10Times();
});
task.Start();
```

The System.Threading.Tasks.Task class

- The following table describes some of the key properties and methods:

Property Name	Description
Status	Gets the TaskStatus of this task
CompletedTask	Gets a task that has already completed successfully
IsCanceled	Gets whether this Task instance has completed execution due to being canceled
IsCompleted	Gets a value that indicates whether the task has completed
IsCompletedSuccessfully	Gets whether the task ran to completion
Method Name	**Description**
ContinueWith(Action<Task>)	Creates a continuation that executes asynchronously when the target Task completes
Run(Action)	Queues the specified work to run on the thread pool and returns a Task object that represents that work
Start()	Starts the Task, scheduling it for execution to the current TaskScheduler
Wait()	Waits for the Task to complete execution
WhenAll(Task[])	Creates a task that will complete when all of the Task objects in an array have completed

Using Task Demonstration - 01

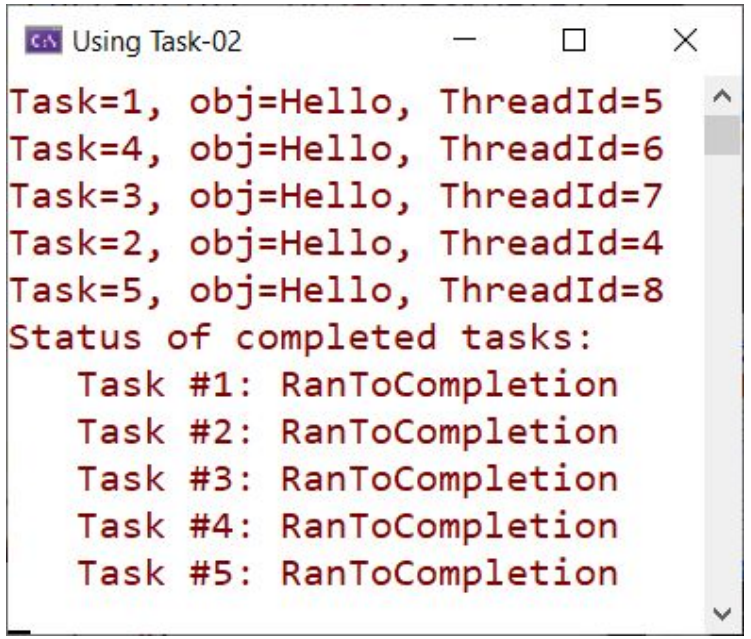

```
using System;
using System.Threading;
using System.Threading.Tasks;

class Program {
    static void PrintNumber(string message){
        for (int i = 1; i <= 5; i++) {
            Console.WriteLine($"{message}:{i}");
            Thread.Sleep(1000);
        }
    }//end PrintNumber
    static void Main() {
        Thread.CurrentThread.Name = "Main";
        // Create a task by using a lambda expression
        Task task01 = new Task(() => PrintNumber("Task 01"));
        task01.Start();
        // Create a task by using delegate and run the task
        Task task02 = Task.Run(delegate {
            PrintNumber("Task 02");
        });
        // Create a task by using a Action delegate
        Task task03 = new Task(new Action(() =>{
            PrintNumber("Task 03");
        }));
        task03.Start();
        Console.WriteLine($"Thread '{Thread.CurrentThread.Name}'");
        Console.ReadKey();
    }//end Main
}//end Program
```

```
Using Task
Task 03:1
Thread 'Main'
Task 02:1
Task 01:1
Task 02:2
Task 03:2
Task 01:2
Task 02:3
Task 03:3
Task 01:3
Task 01:4
Task 03:4
Task 02:4
Task 01:5
Task 03:5
Task 02:5
```

Using Task Demonstration - 02

- The demonstration creates five tasks, waits for all five to complete, and then displays their status

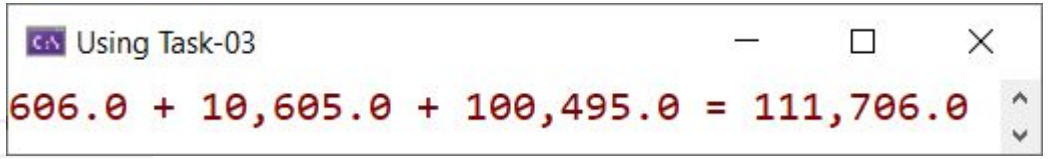
```
class Program{
    public static void Main() {
        // Wait for all tasks to complete.
        Task[] tasks = new Task[5];
        String taskData = "Hello";
        for (int i = 0; i < 5; i++){
            tasks[i] = Task.Run(() =>
            {
                Console.WriteLine($"Task={ Task.CurrentId}, obj={taskData}, " +
                    $"ThreadId={Thread.CurrentThread.ManagedThreadId}");
                Thread.Sleep(1000);
            });
        }
        try{
            Task.WaitAll(tasks);
        }
        catch (AggregateException ae){
            Console.WriteLine("One or more exceptions occurred: ");
            foreach (var ex in ae.Flatten().InnerExceptions)
                Console.WriteLine("   {0}", ex.Message);
        }
        Console.WriteLine("Status of completed tasks:");
        foreach (var t in tasks){
            Console.WriteLine("   Task #{0}: {1}", t.Id, t.Status);
        }
    }//end Main
}//end Program
```



Using Task Demonstration - 03

- The demonstration creates three tasks to calculate the sum, waits for all to complete, and then displays the result

```
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
class Program {
    private static double DoComputation(double start){
        double sum = 0;
        for (var value = start; value <= start + 10; value += .1){
            sum += value;
        }
        return sum;
    }//end DoComputation
    public static void Main() {
        Task<Double>[] taskArray = { Task<double>.Factory.StartNew(() => DoComputation(1.0)),
                                     Task<double>.Factory.StartNew(() => DoComputation(100.0)),
                                     Task<double>.Factory.StartNew(() => DoComputation(1000.0)) };

        var results = new double[taskArray.Length];
        double sum = 0;
        for (int i = 0; i < taskArray.Length; i++){
            results[i] = taskArray[i].Result;
            Console.Write("{0:N1} {1}", results[i],
                          i == taskArray.Length - 1 ? "= " : "+ ");
            sum += results[i];
        }
        Console.WriteLine("{0:N1}", sum);
    }//end Main
}//end Program
```

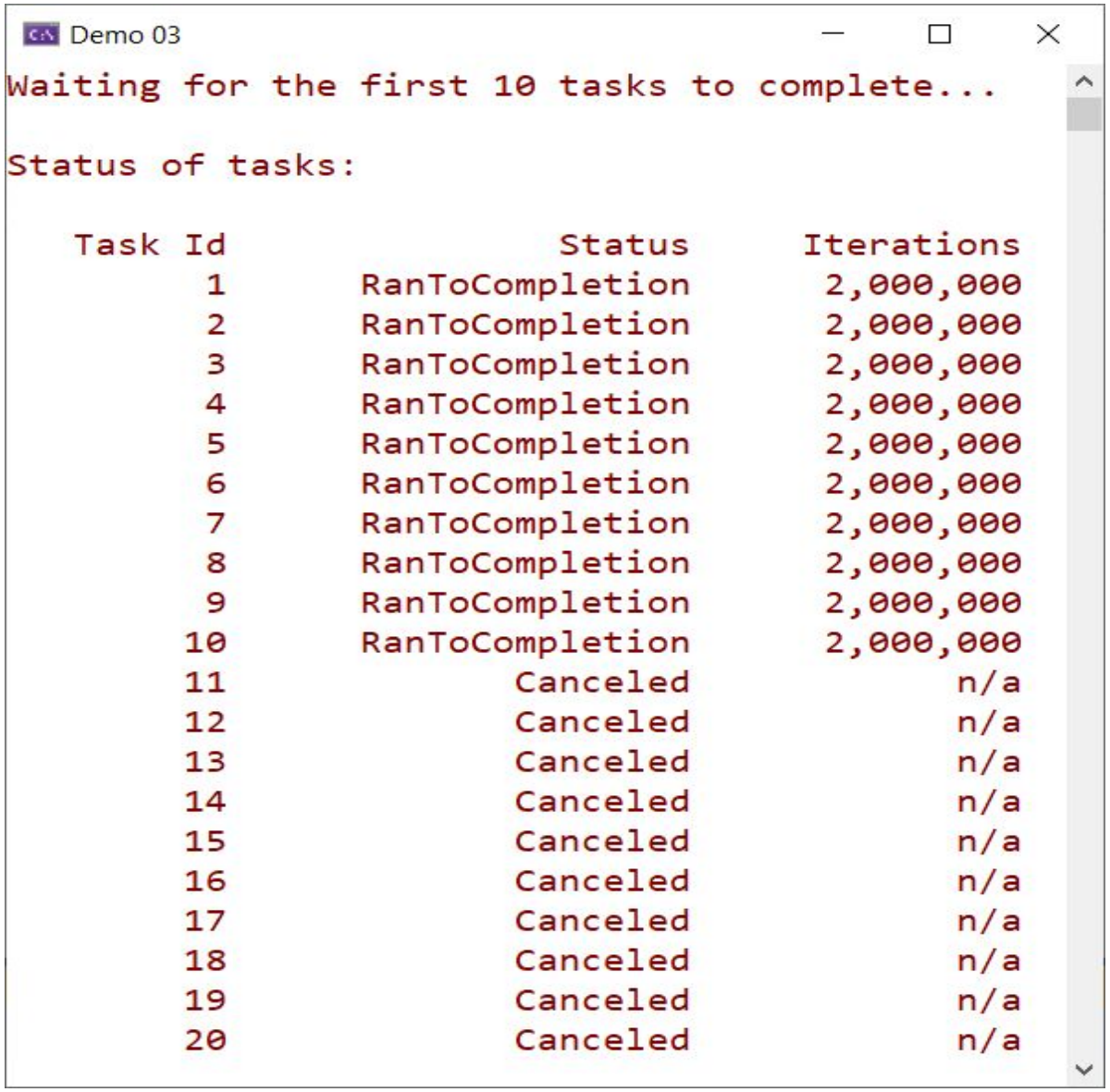


Using Task Demonstration - 04

- The demonstration creates 20 tasks that will loop until a counter is incremented to a value of 2 million. When the first 10 tasks reach 2 million, the cancellation token is cancelled, and any tasks whose counters have not reached 2 million are cancelled

```
class Program
{
    static void Main(string[] args) {
        var tasks = new List<Task<int>>();
        var source = new CancellationTokenSource();
        var token = source.Token;
        int completedIterations = 0;

        for (int n = 1; n <= 20; n++)
        {
            tasks.Add(Task.Run(() =>
            {
                int iterations = 0;
                for (int ctr = 1; ctr <= 2_000_000; ctr++) {
                    token.ThrowIfCancellationRequested();
                    iterations++;
                }
                Interlocked.Increment(ref completedIterations);
                if (completedIterations >= 10)
                    source.Cancel();
                return iterations;
            }, token));
        }
        Console.WriteLine("Waiting for the first 10 tasks to complete...\n");
        try{
            Task.WaitAll(tasks.ToArray());
        }
        catch (AggregateException) {
            Console.WriteLine("Status of tasks:\n");
            Console.WriteLine("{0,10} {1,20} {2,14:N0}", "Task Id",
                              "Status", "Iterations");
            foreach (var t in tasks)
                Console.WriteLine("{0,10} {1,20} {2,14}",
                                  t.Id, t.Status,
                                  t.Status != TaskStatus.Canceled ?
                                  t.Result.ToString("N0") : "n/a");
        }
        Console.ReadLine();
    }//end Main
}//end Program
```

Using Task Demonstration - 04



Understanding Parallel LINQ (PLINQ)

- Parallel LINQ (PLINQ) is a parallel implementation of the Language-Integrated Query (LINQ) pattern. PLINQ implements the full set of LINQ standard query operators as extension methods for the System.Linq namespace and has additional operators for parallel operations
- PLINQ combines the simplicity and readability of LINQ syntax with the power of parallel programming. PLINQ is a parallel implementation of LINQ for objects
- LINQ queries execute sequentially and can be really slow for heavy computing operations. PLINQ supports the parallel execution of queries by having a task scheduled to be run on multiple threads and optionally on multiple cores as well
- .NET supports the seamless conversion of LINQ to PLINQ using the AsParallel() method. PLINQ is a very good choice for computing heavy operations

What is a Parallel Query?

- A PLINQ query in many ways resembles a non-parallel LINQ to Objects query
- PLINQ queries, just like sequential LINQ queries, operate on any in-memory IEnumerable or IEnumerable<T> data source, and have deferred execution, which means they do not begin executing until the query is enumerated
- The primary difference is that PLINQ attempts to make full use of all the processors on the system. It does this by partitioning the data source into segments, and then executing the query on each segment on separate worker threads in parallel on multiple processors
- Through parallel execution, PLINQ can achieve significant performance improvements over legacy code for certain kinds of queries, often just by adding the AsParallel query operation to the data source. However, parallelism can introduce its own complexities, and not all query operations run faster in PLINQ

The ParallelEnumerable Class

- The ParallelEnumerable class is available in the System.Linq namespace and the System.Core assembly
- Apart from supporting most of the standard query operators defined by LINQ, the ParallelEnumerable class supports a lot of extra methods that support parallel execution:

ParallelEnumerable Operator	Description
AsParallel	The entry point for PLINQ. Specifies that the rest of the query should be parallelized, if it is possible
AsSequential	Specifies that the rest of the query should be run sequentially, as a non-parallel LINQ query
AsOrdered	Specifies that PLINQ should preserve the ordering of the source sequence for the rest of the query, or until the ordering is changed, for example by the use of an orderby (Order By in Visual Basic) clause

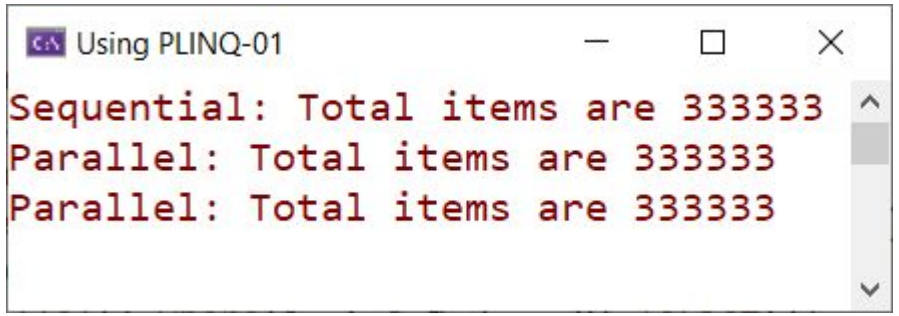
The ParallelEnumerable Class

ParallelEnumerable Operator	Description
AsUnordered	Specifies that PLINQ for the rest of the query is not required to preserve the ordering of the source sequence
ForAll	A multithreaded enumeration method that, unlike iterating over the results of the query, enables results to be processed in parallel without first merging back to the consumer thread
Aggregate overload	An overload that is unique to PLINQ and enables intermediate aggregation over thread local partitions, plus a final aggregation function to combine the results of all partitions
WithDegreeOfParallelism	Specifies the maximum number of processors that PLINQ should use to parallelize the query
WithMergeOptions	Provides a hint about how PLINQ should, if it is possible, merge parallel results back into just one sequence on the consuming thread
WithExecutionMode	Specifies whether PLINQ should parallelize the query even when the default behavior would be to run it sequentially

Using PLINQ Demonstration - 01

- The demonstration finds all the numbers that are divisible by three

```
using System;
using System.Linq;
using System.Threading.Tasks;
class Program
{
    public static void Main()
    {
        var range = Enumerable.Range(1, 1000_000);
        //Here is Sequential version
        var resultList = range.Where(i => i % 3 == 0).ToList();
        Console.WriteLine($"Sequential: Total items are {resultList.Count}");
        //Here is Parallel version using .AsParallel method
        resultList = range.AsParallel().Where(i => i % 3 == 0).ToList();
        Console.WriteLine($"Parallel: Total items are {resultList.Count}");
        resultList = (from i in range.AsParallel()
                      where i % 3 == 0
                      select i).ToList();
        Console.WriteLine($"Parallel: Total items are {resultList.Count}");
        Console.ReadLine();
    }//end Main
}//end Program
```



Using PLINQ Demonstration - 02

- This example demonstrates Parallel.ForEach for CPU intensive operations. The application randomly generates 2 million numbers and tries to filter to prime numbers. The first case iterates over the collection via a for loop. The second case iterates over the collection via Parallel.ForEach. The resulting time taken by each iteration is displayed when the application is finished

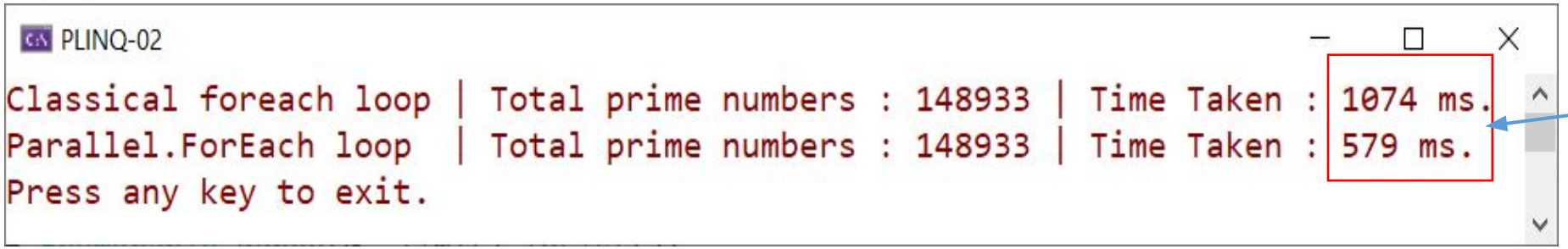
```
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
using System.Threading.Tasks;
class Program {
    // IsPrime returns true if number is Prime, else false.
    private static bool IsPrime(int number) {
        bool result = true;
        if (number < 2) {
            return false;
        }
        for (var divisor = 2; divisor <= Math.Sqrt(number) && result == true; divisor++){
            if (number % divisor == 0) {
                result = false;
            }
        }
        return result;
    }//end IsPrime
```

Using PLINQ Demonstration - 02

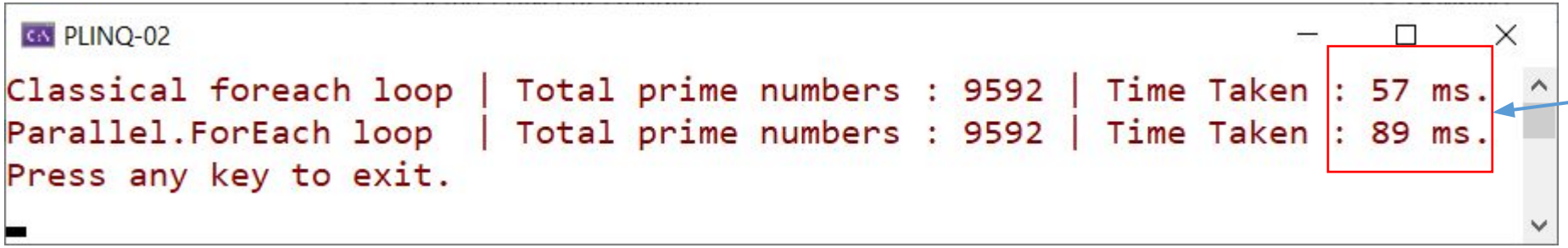

```
//GetPrimeList returns Prime numbers by using sequential ForEach
private static IList<int> GetPrimeList(IList<int> numbers) => numbers.Where(IsPrime).ToList();

//GetPrimeListWithParallel returns Prime numbers by using Parallel.ForEach
private static IList<int> GetPrimeListWithParallel(IList<int> numbers) {
    var primeNumbers = new ConcurrentBag<int>();
    Parallel.ForEach(numbers, number => {
        if (IsPrime(number)){
            primeNumbers.Add(number);
        }
    });
    return primeNumbers.ToList();
}//end GetPrimeListWithParallel
static void Main(){
    // 2 million
    var limit = 2_000_000;
    var numbers = Enumerable.Range(0, limit).ToList();

    var watch = Stopwatch.StartNew();
    var primeNumbersFromForeach = GetPrimeList(numbers);
    watch.Stop();

    var watchForParallel = Stopwatch.StartNew();
    var primeNumbersFromParallelForeach = GetPrimeListWithParallel(numbers);
    watchForParallel.Stop();

    Console.WriteLine($"Classical foreach loop | Total prime numbers :" +
        $" {primeNumbersFromForeach.Count} | Time Taken : " +
        $"{watch.ElapsedMilliseconds} ms.");
```

Using PLINQ Demonstration - 02


```
        Console.WriteLine($"Parallel.ForEach loop  | Total prime numbers : " +
            $"{primeNumbersFromParallelForeach.Count} | Time Taken : " +
            $"{watchForParallel.ElapsedMilliseconds} ms.");

        Console.WriteLine("Press any key to exit.");
        Console.ReadLine();
    }//end Main
}//end Program
```



With 2.000.000 numbers



With 100.000 numbers

Disadvantages of Parallel Programming with PLINQ

In most cases, PLINQ performs much faster than its non-parallel counterpart LINQ. However, there is some performance overhead, which is related to partitioning and merging while parallelizing the LINQ. The following are some of the things we need to consider while using PLINQ:

- **Parallel is not always faster**: Parallelization is an overhead. Unless our source collection is huge or it has compute-bound operations, it makes more sense to execute the perations in sequence. Always measure the performance of sequential and parallel queries to make an informed decision
- **Avoid I/O operations that involve atomicity**: All I/O operations that involve writing to a filesystem, database, network, or shared memory location should be avoided inside PLINQ. This is because these methods are not thread-safe, so using them may lead to exceptions. A solution would be to use synchronization primitives, but this would also reduce performance drastically
- **Queries may not always be running in parallel**: Parallelization in PLINQ is a decision that's taken by Core CLR. Even if we called the AsParallel() method in the query, it isn't guaranteed to take a parallel path and may run sequentially instead

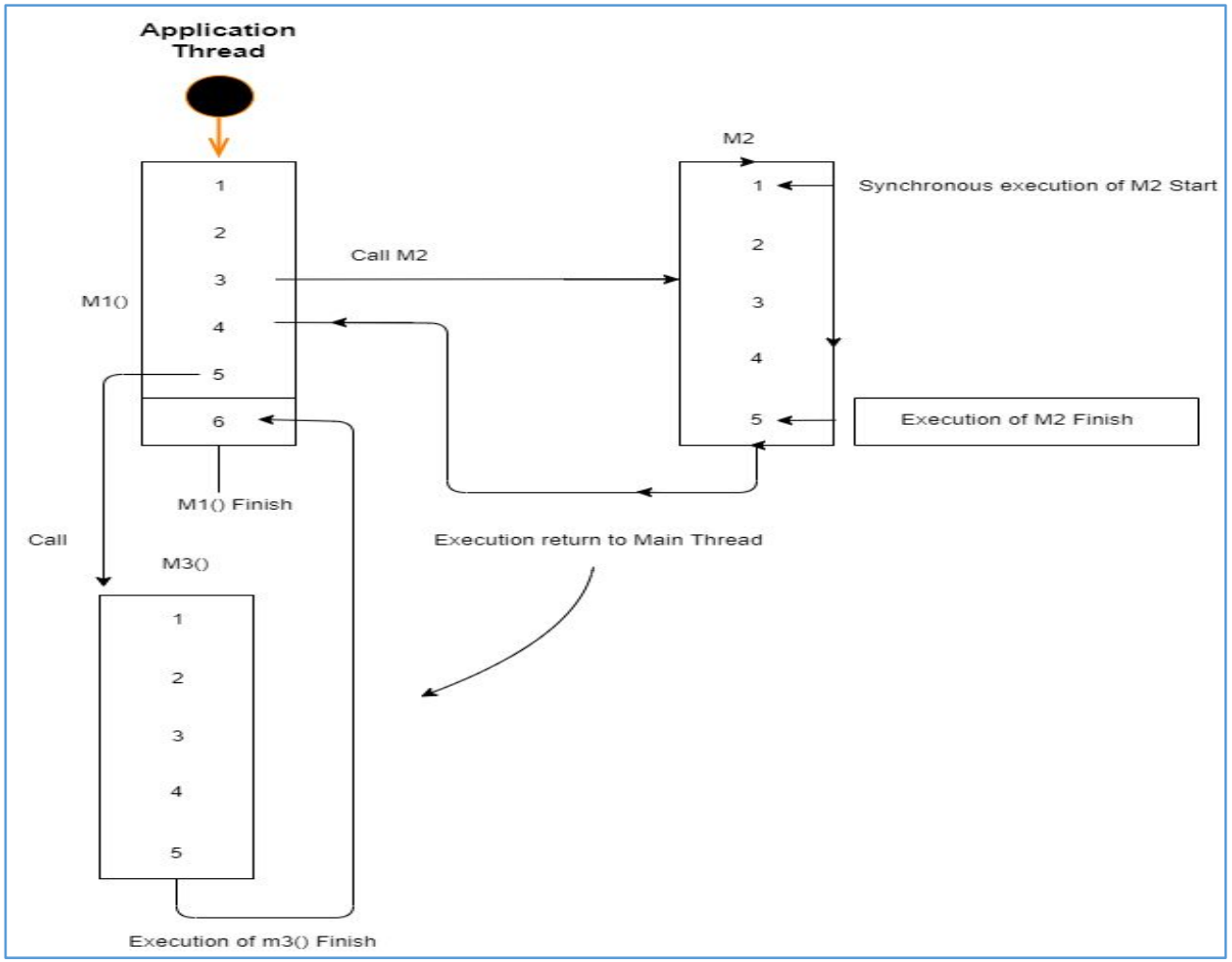
Asynchronous Programming in .NET

Understanding Synchronous Program Execution

- In the case of synchronous execution, control never moves out of the calling thread. Code is executed one line at a time, and, when a function is called, the calling thread waits for the function to finish executing before executing the next line of code
- Synchronous programming is the most commonly used method of programming and it works well due to the increase in CPU performance. With faster processors, the code completes sooner
- With parallel programming, we have seen that we can create multiple threads that can run concurrently. We can start many threads but also make the main program flow synchronous by calling structures such as Thread.Join and Task.Wait. An example of synchronous code as follows:

Understanding Synchronous Program Execution

1. We start the application thread by calling the M1() method
2. At line 3, M1() calls M3() synchronously.
3. The moment the M2() method is called, the control execution transfers to the M1() method
4. Once the called method (M2) is finished, the control returns to the main thread, which executes the rest of the code in M1(), that is, lines 4 and 5
5. The same thing happens on line 5 with a call to M2. Line 6 executes when M2 has finished



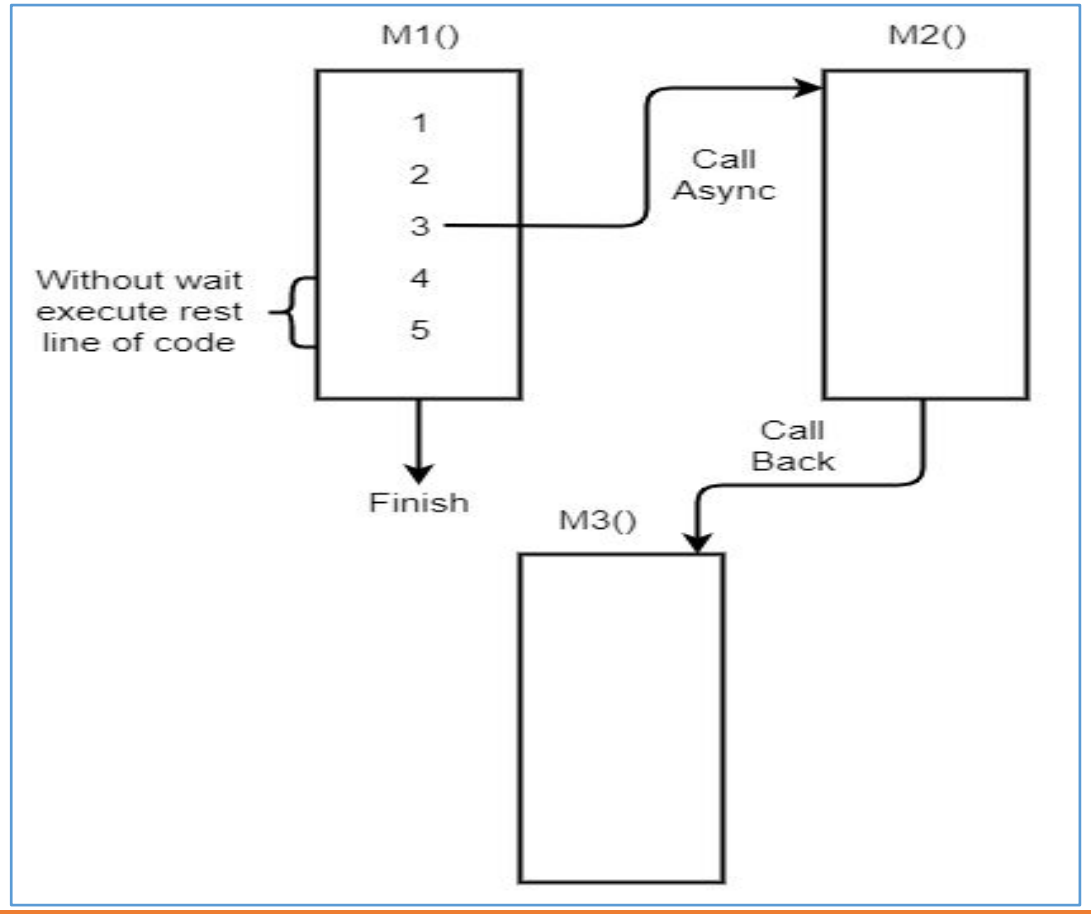
Understanding Asynchronous Program Execution

- The asynchronous model allows us to execute multiple tasks concurrently. If we call a method asynchronously, the method is executed in the background while the thread that is called returns immediately and executes the next line of code
- The asynchronous method may or may not create a thread, depending on the type of task we're dealing with
- When the asynchronous method finishes, it returns the result to the program via callbacks. An asynchronous method can be void, in which case we don't need to specify callbacks
- The patterns were supported to perform I/O bound and compute-bound operations by .NET:
  - Asynchronous Programming Model (**APM**): Using Delegate.BeginInvoke is no longer supported in .NET Core
  - Event-Based Asynchronous Pattern (**EAP**) and The Task-Based Asynchronous Pattern (**TAP**)

Understanding Asynchronous Program Execution

- The example of asynchronous code as follows:

1. While executing M1(), the caller thread makes asynchronous calls to M2()
2. The caller thread provides a callback function, say, M3(), while calling M2()
3. The caller thread doesn't wait for M2() to finish; instead, it finishes the rest of the code in M1() (if there is any to finish)
4. M2() will be executed by the CPU either instantly in a separate thread or at a later date (period)
5. Once M2() finishes, M3() is called, which receives output from M2() and processes it



Asynchronous Demonstration - 01

- This example demonstrates using Event-Based Asynchronous Pattern (EAP) to download a web page

```
using System;
using System.Net;
class Program {
    //Using Event-Based Asynchronous Pattern(EAP)
    private static void DownloadAsynchronously(){
        WebClient client = new WebClient();
        client.DownloadStringCompleted +=
            new DownloadStringCompletedEventHandler(DownloadComplete);
        client.DownloadStringAsync(new Uri("http://www.aspnet.com"));
    }
    //--------------------------------------------------------
    private static void DownloadComplete(object sender,
        DownloadStringCompletedEventArgs e) {
        if (e.Error != null) {
            Console.WriteLine("Some error has occured.");
            return;
        }
        //Print result
        Console.WriteLine(e.Result);
        Console.WriteLine(new string('*', 30));
        Console.WriteLine("Download completed.");
    }
    //--------------------------------------------------------
    static void Main(string[] args){
        DownloadAsynchronously();
        Console.WriteLine("Main thread : Done");
        Console.WriteLine(new string('*',30));
        Console.ReadLine();
    }//end Main
}//end Program
```

```
Using Event-Based Asynchronous Pattern(EAP
Main thread : Done
******************************
<!DOCTYPE html><html lang="en"  data-adblockkey=MF
wwDQYJKoZIhvcNAQEBBQADSwAwSAJBANnylWw2vLY4hUn9w06z
QKbhKBfvjFUCsdFlb6TdQhxb9RXWXuI4t31c+o8fYOv/s8q1LG
Pga3DE1L/tHU4LENMCAwEAAQ==_ApCdwDziDW0ZWEX9JDcgQZM
mkP74MMYeNEToJX/crN/i/N1tBHPWnpaVyoSTzQse58Lc2sqQi
ztRZFWHOe9Sjw==><head><meta charset="utf-8"><title
>aspnet.com -&nbspThis website is for sale!&n
bsp;-&nbspaspnet Resources and Information.</title
><meta name="viewport" content="width=device-width
,initial-scale=1.0,maximum-scale=1.0,user-scalable
=0"><meta name="description" content="This website
```

When to use Asynchronous Programming

- There are many situations in which Direct Memory Access (DMA) is used to access the host system or I/O operations (such as files, databases, or network access) are used, which is where processing is done by the CPU rather than the application thread

- In the preceding scenario, the calling thread makes a call to the I/O API and waits for the task to complete by moving to a blocked state. When the task is completed by the CPU, the thread is unblocked and finishes the rest of the method

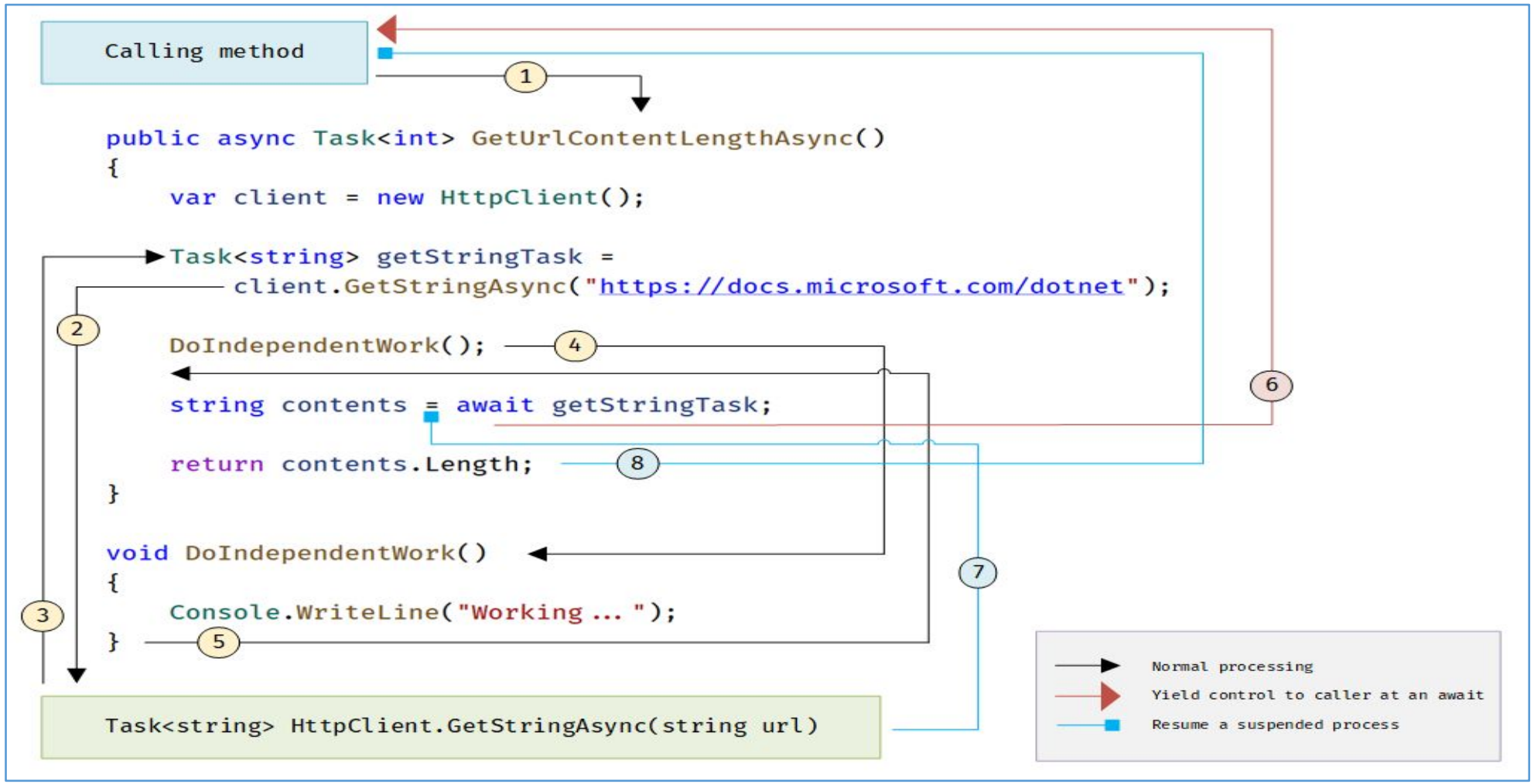
- Using asynchronous methods, we can improve the application's performance and responsiveness. We can also execute a method via a different thread

Introducing async and await

- **async** and **await** are two very popular keywords among .NET Core developers writing asynchronous code with the new asynchronous APIs provided by .NET
- The **async** and **await** keywords in C# are the heart of async programming. By using those two keywords, we can use resources in .NET Framework, .NET Core, or the Windows Runtime to create an asynchronous method almost as easily as we create a synchronous method
- Asynchronous methods that we define by using the **async** keyword are referred to as **async methods**

```
public async Task<int> GetUrlContentLengthAsync() {
  var client = new HttpClient();
  Task<string> getStringTask =
       client.GetStringAsync("https://docs.microsoft.com/dotnet");
  DoIndependentWork();
  string contents = await getStringTask;
  return contents.Length;
}
void DoIndependentWork() {
  Console.WriteLine("Working...");
}
```

Introducing async and await



The diagram shows what happens in an async method

Asynchronous Demonstration - 02

- This example demonstrates using Task-Based Asynchronous Pattern (TAP)

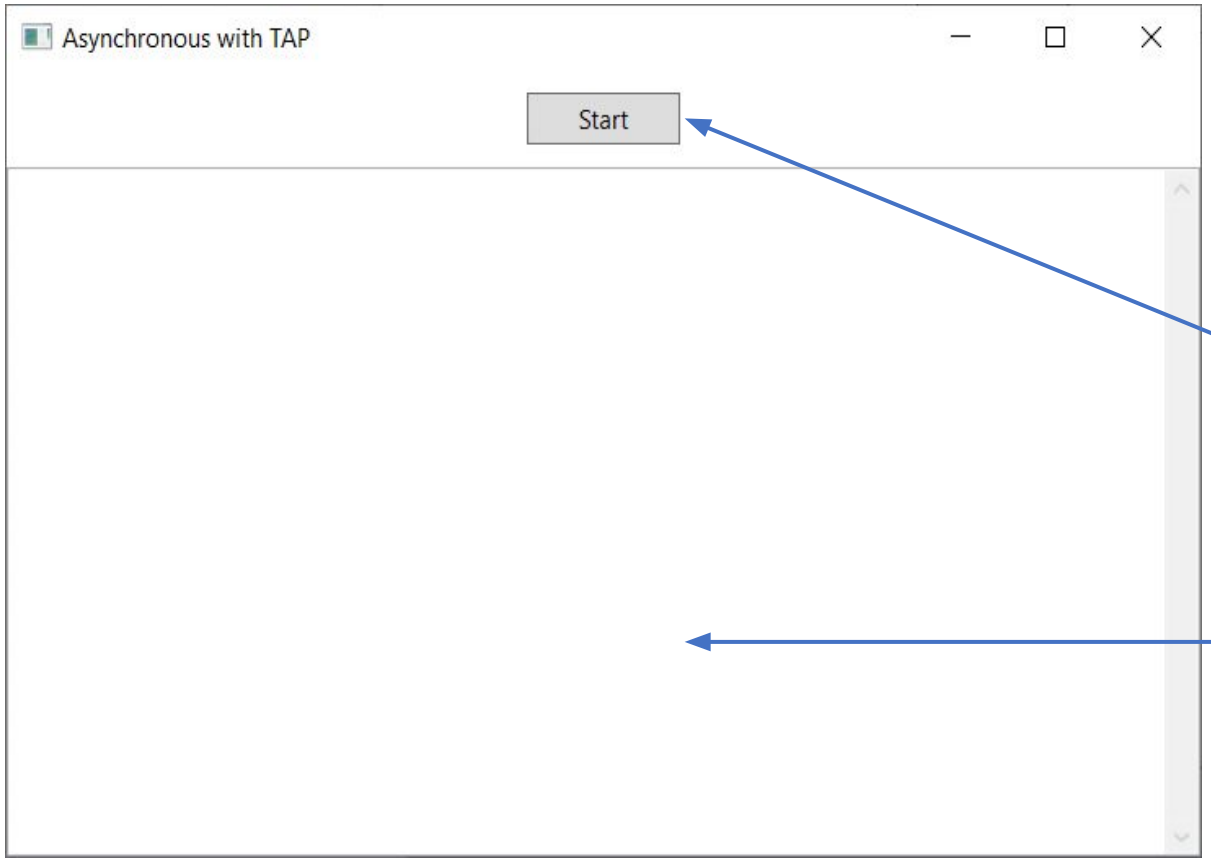
```
using System;
using System.Threading.Tasks;
class Program{
    public static async Task<int> Method1() {
        int count = 0;
        await Task.Run(() =>{
            for (int i = 1; i <= 10; i++) {
                Console.WriteLine("Method 1");
                count += 1;
            }
        });
        return count;
    }
    //----------------------------------------
    public static void Method2(){
        for (int i = 1; i <= 5; i++){
            Console.WriteLine("Method 2");
        }
    }
    //----------------------------------------
    public static void Method3(int count){
        Console.WriteLine("Method 3 is called.");
        Console.WriteLine($"Total count is {count}");
    }
    //----------------------------------------
    public static async Task callMethod(){
        Method2();
        var count = await Method1();
        Method3(count);
    }
    //----------------------------------------
    static async Task Main(string[] args)
    {
        await callMethod();
        Console.ReadKey();
    }//end Main
}//end Program
```

```
Using TAP
Method 2
Method 2
Method 2
Method 2
Method 2
Method 1
Method 1
Method 1
Method 1
Method 1
Method 1
Method 1
Method 1
Method 1
Method 1
Method 3 is called.
Total count is 10
```

Asynchronous Demonstration - 03

- This example demonstrates using Task-Based Asynchronous Pattern (TAP) with HttpClient to download the contents of a website in the WPF application

1.Create a WPF app named **AsyncExample** with UI as follows :




```
<Window x:Class=...
//xmlns=
//...
Title="Asynchronous with TAP" Height="400" Width="600"
MinHeight="300" MinWidth="500">
    <Grid>
        <Button x:Name="btnStartButton"
          Content="Start" HorizontalAlignment="Center"
          Margin="0,10,0,0" VerticalAlignment="Top"
          Width="75" Height="24"
          Click="OnStartButtonClick" />
        <TextBox x:Name="txtResults" TextWrapping="Wrap"
                 FontFamily="Consolas"
                 VerticalScrollBarVisibility="Visible"
                 Margin="0,45,0,0" />
    </Grid>
</Window>
```

XAML code of MainWindow.xaml

Asynchronous Demonstration - 03

2. Write codes in **MainWindow.xaml.cs** as follows:

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Net.Http;
using System.Threading.Tasks;
using System.Windows;

public partial class MainWindow : Window {
    private readonly HttpClient client = new HttpClient {
     MaxResponseContentBufferSize = 1_000_000
    };
    //-------------------------------------------------
    private readonly IEnumerable<string> UrlList = new string[] {
        "https://docs.microsoft.com",
        "https://docs.microsoft.com/azure",
        "https://docs.microsoft.com/powershell",
        "https://docs.microsoft.com/dotnet",
        "https://docs.microsoft.com/aspnet/core",
        "https://docs.microsoft.com/windows"
    };
    //-------------------------------------------------
    private async void OnStartButtonClick(object sender, RoutedEventArgs e) {
        btnStartButton.IsEnabled = false;
        txtResults.Clear();
        await SumPageSizesAsync();
        txtResults.Text += $"\nControl returned to {nameof(OnStartButtonClick)}.";
        btnStartButton.IsEnabled = true;
    }
```

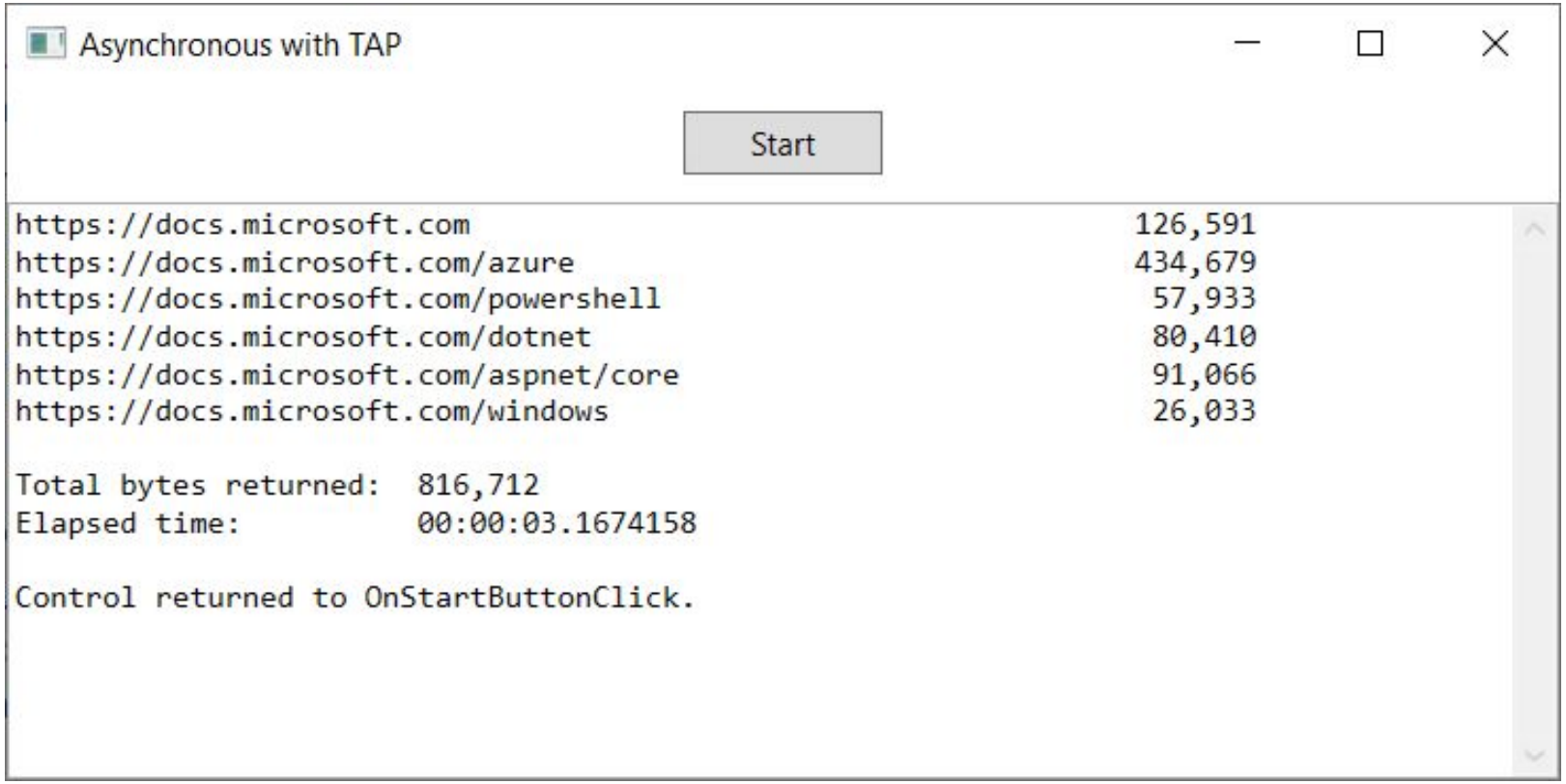
```
"https://docs.microsoft.com",
"https://docs.microsoft.com/azure",
"https://docs.microsoft.com/powershell",
"https://docs.microsoft.com/dotnet",
"https://docs.microsoft.com/aspnet/core",
"https://docs.microsoft.com/windows"
```

Asynchronous Demonstration - 03


```
    //-------------------------------------------------
    private async Task SumPageSizesAsync() {
        var stopwatch = Stopwatch.StartNew();
        int total = 0;
        foreach (string url in UrlList) {
            int contentLength = await ProcessUrlAsync(url, client);
            total += contentLength;
        }
        stopwatch.Stop();
        txtResults.Text += $"\nTotal bytes returned:  {total:#,#}";
        txtResults.Text += $"\nElapsed time:          {stopwatch.Elapsed}\n";
    }
    //-------------------------------------------------
    private async Task<int> ProcessUrlAsync(string url, HttpClient client){
        byte[] content = await client.GetByteArrayAsync(url);
        DisplayResults(url, content);
        return content.Length;
    }
    //-------------------------------------------------
    private void DisplayResults(string url, byte[] content) =>
        txtResults.Text += $"{url,-60} {content.Length,10:#,#}\n";
    //-------------------------------------------------
    protected override void OnClosed(EventArgs e) => client.Dispose();
}//end class
```

Asynchronous Demonstration - 03

3. Press **Ctrl+F5** to run project and press **Start** button to view the output



Summary

- Concepts were introduced:
  - Overview Mono-Processor Systems and Multiprocessor Systems
  - Overview Multiple Core Processors and Hyper-Threading
  - Overview Flynn's Taxonomy
  - Describe about Serial Computing
  - Describe about Parallel Computing and Types of Parallelism
  - Overview The Parallel Programming Architecture
  - Overview Task Parallel Library (TPL) and Parallel LINQ (PLINQ)
  - Overview Asynchronous Programming in .NET
  - Demo about Task Parallel Library (TPL) and Parallel LINQ (PLINQ)
  - Demo about Asynchronous Programming by async and await keywords